```
# Titanic Survival Prediction Lab

## This lab explores machine learning models to predict passenger survival on the Titanic dataset.
## We begin with the reference **KNN model**, then extend the analysis by adding **Logistic Regression** and **D
## The goal is to compare performance across models, interpret results, and highlight trade-offs between accurac
## Both training and test datasets are used to ensure fair evaluation.
```

```
# ==================++++++++++++++++++++++++++++
# Titanic Survival Lab - Classification Models
# Author Mind : Kanishk Sharma
# Date: March-16th 2026
# ==================+++++++++++++++++++++++++++

# Step 1: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
from google.colab import files

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# ==================++++++++++++++++++++++++++++
# Step 2: Load Training and Test Datasets
#==================+++++++++++++++++++++++++++

train_file_name_expected = "train_KNN.csv"
test_file_name_expected = "test_KNN.csv"

# Initialize actual file names, these will be updated if files are uploaded
train_file_name = train_file_name_expected
test_file_name = test_file_name_expected

# Check if files exist, if not, prompt for upload
if not os.path.exists(train_file_name_expected) or not os.path.exists(test_file_name_expected):
    print(f"Required files '{train_file_name_expected}' and '{test_file_name_expected}' not found.")
    print("Please upload the files.")
    uploaded = files.upload()

    # Determine the actual names of the uploaded files
    actual_train_file_name = None
    actual_test_file_name = None

    for fname in uploaded.keys():
        if train_file_name_expected.replace('.csv', '') in fname: # Check for variations like 'train_KNN (1).csv
            actual_train_file_name = fname
        if test_file_name_expected.replace('.csv', '') in fname: # Check for variations like 'test_KNN (1).csv'
            actual_test_file_name = fname

    if actual_train_file_name:
        train_file_name = actual_train_file_name
        print(f"Using uploaded file for training: '{train_file_name}'")
    else:
        print(f"Error: Could not find '{train_file_name_expected}' or a variation in uploaded files. Please ensu
        raise FileNotFoundError(f"Missing required file: '{train_file_name_expected}'")

    if actual_test_file_name:
        test_file_name = actual_test_file_name
```

```
        test_file_name = actual_test_file_name
        print(f"Using uploaded file for testing: '{test_file_name}'")
    else:
        print(f"Error: Could not find '{test_file_name_expected}' or a variation in uploaded files. Please ensur
        raise FileNotFoundError(f"Missing required file: '{test_file_name_expected}'")

train_df = pd.read_csv(train_file_name)
test_df = pd.read_csv(test_file_name)

print("Training set shape:", train_df.shape)
print("Test set shape:", test_df.shape)

# Preview first few rows
train_df.head()

# ==================++++++++++++++++++++++++++
# Step 3: Data Cleaning and Preprocessing
# ==================++++++++++++++++++++++++++

# Drop irrelevant columns
cols_to_drop = ["PassengerId","Name","Ticket","Cabin"]
train_df = train_df.drop(cols_to_drop, axis=1)
test_df = test_df.drop(cols_to_drop, axis=1)

# Handle missing values
train_df["Age"].fillna(train_df["Age"].median(), inplace=True)
test_df["Age"].fillna(test_df["Age"].median(), inplace=True)

train_df["Embarked"].fillna(train_df["Embarked"].mode()[0], inplace=True)
test_df["Embarked"].fillna(test_df["Embarked"].mode()[0], inplace=True)

# Handle missing Fare values, typically only in test set for Titanic
train_df["Fare"].fillna(train_df["Fare"].median(), inplace=True) # Fill any missing in train_df if present
test_df["Fare"].fillna(train_df["Fare"].median(), inplace=True) # Use train median for test set to prevent data

# Encode categorical variables
train_df = pd.get_dummies(train_df, drop_first=True)
test_df = pd.get_dummies(test_df, drop_first=True)

# Align train and test columns
train_df, test_df = train_df.align(test_df, join="left", axis=1, fill_value=0)

# Separate features and target
X_train = train_df.drop("Survived", axis=1)
y_train = train_df["Survived"]

X_test = test_df.drop("Survived", axis=1)
y_test = test_df["Survived"]

print("Processed training shape:", X_train.shape)
print("Processed test shape:", X_test.shape)

# ==================++++++++++++++++++++++++++
# Step 4: Feature Scaling
# ==================++++++++++++++++++++++++++

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaling complete. Ready for model training.")

#==================++++++++++++++++++++++++++
# Step 5: Train Models
# ==================++++++++++++++++++++++++++

# --- KNN ---
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
```

```
knn_preds = knn.predict(X_test_scaled)

# --- Logistic Regression ---
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_scaled, y_train)
log_preds = log_reg.predict(X_test_scaled)

# --- Decision Tree ---
dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_train, y_train)  # Trees don’t require scaling
dt_preds = dt.predict(X_test)

#==================++++++++++++++++++++++++++
# Step 6: Evaluation Function
# ==================++++++++++++++++++++++++++

def evaluate_model(name, y_true, y_pred):
    print(f"\n--- {name} ---")
    print("Accuracy:", accuracy_score(y_true, y_pred))
    print("Classification Report:\n", classification_report(y_true, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))

# Evaluate all models
evaluate_model("KNN", y_test, knn_preds)
evaluate_model("Logistic Regression", y_test, log_preds)
evaluate_model("Decision Tree", y_test, dt_preds)

# ==================++++++++++++++++++++++++++
# Step 7: Visualizations
# ==================++++++++++++++++++++++++++

# Logistic Regression Coefficients
coefficients = pd.Series(log_reg.coef_[0], index=X_train.columns)
plt.figure(figsize=(10,6))
coefficients.sort_values().plot(kind='barh', color='skyblue')
plt.title("Logistic Regression Coefficients (Feature Impact)")
plt.xlabel("Coefficient Value")
plt.ylabel("Features")
plt.show()

# Decision Tree Feature Importance
importances = pd.Series(dt.feature_importances_, index=X_train.columns)
plt.figure(figsize=(10,6))
sns.barplot(x=importances, y=importances.index, palette="viridis")
plt.title("Decision Tree Feature Importance")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.show()

# Confusion Matrix Heatmaps
def plot_confusion_matrix(y_true, y_pred, title):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap="Blues",
                xticklabels=["Not Survived","Survived"],
                yticklabels=["Not Survived","Survived"])
    plt.title(title)
    plt.ylabel("Actual")
    plt.xlabel("Predicted")
    plt.show()

plot_confusion_matrix(y_test, knn_preds, "KNN Confusion Matrix")
plot_confusion_matrix(y_test, log_preds, "Logistic Regression Confusion Matrix")
plot_confusion_matrix(y_test, dt_preds, "Decision Tree Confusion Matrix")

# ==================++++++++++++++++++++++++++
# Step 8: Comparison Table
# ==================++++++++++++++++++++++++++
```

```
results = {
    "Model": ["KNN", "Logistic Regression", "Decision Tree"],
    "Accuracy": [
        accuracy_score(y_test, knn_preds),
        accuracy_score(y_test, log_preds),
        accuracy_score(y_test, dt_preds)
    ]
}

results_df = pd.DataFrame(results)
print(results_df)
```